

Technical Specification: Simple-VectorDB Engine Extension (Phase 4)

1. Core Architectural Constraints & Memory Policy

The following parameters and low-level constraints are non-negotiable foundations for this systems phase:

- **Buffer Isolation (\$K\$ Blocks):** All vector workloads—including sequential scans, sorting, and graph indexing—must execute strictly within the bounds set by the `SETBUFFER K` global session manager ($2 \leq K \leq 10$).
- **The Two-Block Active Window:** For basic operations, at most two data blocks may be resident in-memory simultaneously from any underlying binary file. Additional allocations must rely on pin/unpin semantics via the `bufferManager` to avoid memory boundary overflows.
- **Page Granularity Concurrency:** Concurrency control metadata must be evaluated at page granularity before any read or write block access via a custom `LockManager` wrapper.
- **In-Place Matrix Mutability:** Vector re-indexing or re-ordering operations must take place in-place within the temporal files directory, preserving the original data layout unless an explicit `EXPORT` query is executed.

2. High-Dimensional Vector Storage & Binary Layouts

High-dimensional datasets are structured using discrete metadata dimensions and primary vector blocks, ensuring unified page layouts.

2.1 Vector File Layouts

Each high-dimensional dataset is represented as a binary structure split into fixed blocks matching the global `BLOCK_SIZE`.

- **Dense Matrix Files (.matrix):** Stores raw dense float vectors. The initial section of the file maps structural details: global dimension size (D), integer record length (N), and internal type markers.
- **Vector Slot Format:** Inside a page, vector slots are tightly packed. Each record contains:
 - `RecordID` (4-byte unsigned integer)
 - `Vector_Data` ($D \times 4$ bytes, representing an array of 32-bit single-precision IEEE floating-point values)

$$\text{Slot Size (Bytes)} = 4 + (D \times 4)$$

- **Page Structuring Rules:** If a vector slot spans beyond a single **BLOCK_SIZE** boundary, the engine must handle segmented slot crossing using specialized pointer redirections inside the page layout header.

2.2 Vector Graph HNSW Files (.hnsw)

For accelerated proximity processing, graph relationships are stored using specialized disk representations.

- **Node Blocks:** Store the coordinate tracking elements linking a **NodeID** to its baseline **RecordID** in the dense matrix file.
- **Adjacency List Blocks:** Directed multi-layer connections are laid out sequentially on disk page arrays. Each edge record specifies:

\$\$\text{Edge Layout} = \angle \text{Src_NodeID}, \text{Dest_NodeID}, \text{Distance (Float)} \\ \angle\$\$

3. Query Engine Extensions

3.1 LOAD VECTOR MATRIX

- **Syntax:** **LOAD VECTOR MATRIX** <matrix_name> <dimension>
- **Mechanics:** Reads a flat file containing high-dimensional float entries from the **/data** directory. The engine converts the inputs into packed binary pages inside the **/data/temp** workspace.
- **System Controls:** Enforces semantic checks verifying the source presence and prevents conflicting matrix names within the active relational catalog.

3.2 ANNS / KNN QUERY (Approximate Nearest Neighbors)

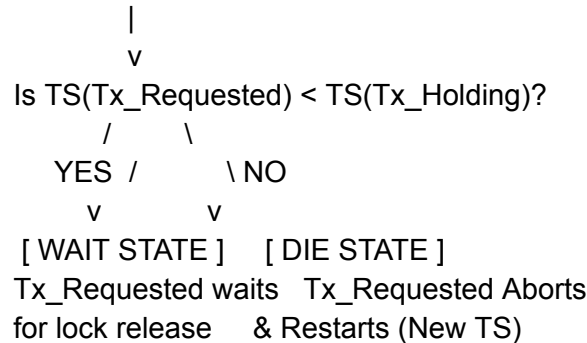
- **Syntax:** **Result <- KNN** <matrix_name> **QUERY_VEC** [<f32_array>] **TOP** <X> **METRIC** <COSINE|EUCLIDEAN>
- **Mechanics:** Computes distances between the target array and disk-resident blocks.
- **Execution Strategy:**
 1. Maintains an internal min-heap tracking the top \$X\$ closest historical elements across pages.
 2. Pulls dense blocks sequentially via the **bufferManager** utilizing a maximum of two concurrent page mappings.
 3. Outputs coordinates directly into a structured output table (**Result**) containing the primary **RecordID** alongside the computed similarity score.

3.3 CONCURRENT VECTOR TRANSACTION

- **Syntax:** **TRANSACTION** <schedule_file>.txt
- **Mechanics:** Processes a sequential transactional log simulating simultaneous page operations.
- **Locking & Deadlock Isolation Rules:**

- Enforces **Strict Two-Phase Locking (2PL)** for page transactions. Reads require shared lock reservations, while vector block mutations require exclusive access.
- Triggers the **Wait-Die Deadlock Prevention Protocol** during lock conflicts. Older transactions are allowed to block and wait for younger allocations; younger transactions instantly abort and initiate a rollback routine.

Lock Conflict Detected (Tx_Requested vs Tx_Holding)



- **Logging Metrics:** The system outputs execution events directly into a file tracking state adjustments:

Plaintext

BEGIN T1 [TS=1]

BEGIN T2 [TS=2]

T1 requests shared lock(matrix_page 4) -> Granted

T2 requests exclusive lock(matrix_page 4) -> T2 dies (younger than T1)

ABORT T2

ROLLBACK T2

COMMIT T1

unlock(matrix_page 4) by T1

RESTART T2 as T2' [TS=5]

4. Evaluative System Reports (**Report4.md**)

The technical submission dossier must include an architectural overview containing:

- **Binary Layout Optimization:** Mathematical justifications demonstrating how your chosen block spacing structures minimize read/write disk line overhead during distance evaluations.
- **Cache Eviction Alignment:** Detailed breakdowns showing how the LRU block replacement mechanics interact with k -NN execution paths to maximize row hits when the buffer size is constrained to a minimum $K=2$.
- **State Machine Correctness:** Trace histories verifying transaction isolation, lock release safety upon rollback transitions, and absolute race stability across simulated execution runs

